Scene Design

Scene Design

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

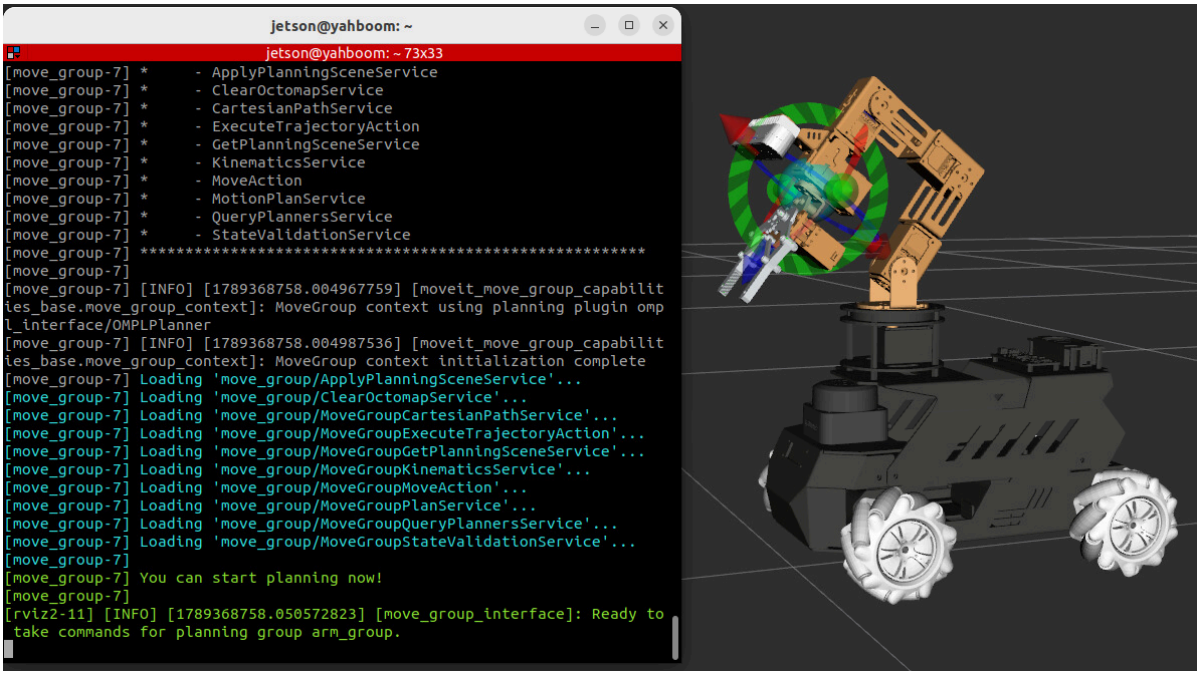
1. Course Content

- This course implements a virtual scene: a three-layer shelf is built, and after the robot arm grasps a cylindrical object, it plans to the middle upper and lower layers to simulate the action of placing the object, while avoiding the shelf obstacles.

2. Preparation

```
ros2 launch yahboomcar_bringup moveit_simulation.launch.py sim2real:=true
rviz_config:=demo
```

Once the program starts, when **You can start planning now!** appears in the terminal, it means the startup was successful, as shown in the figure below:

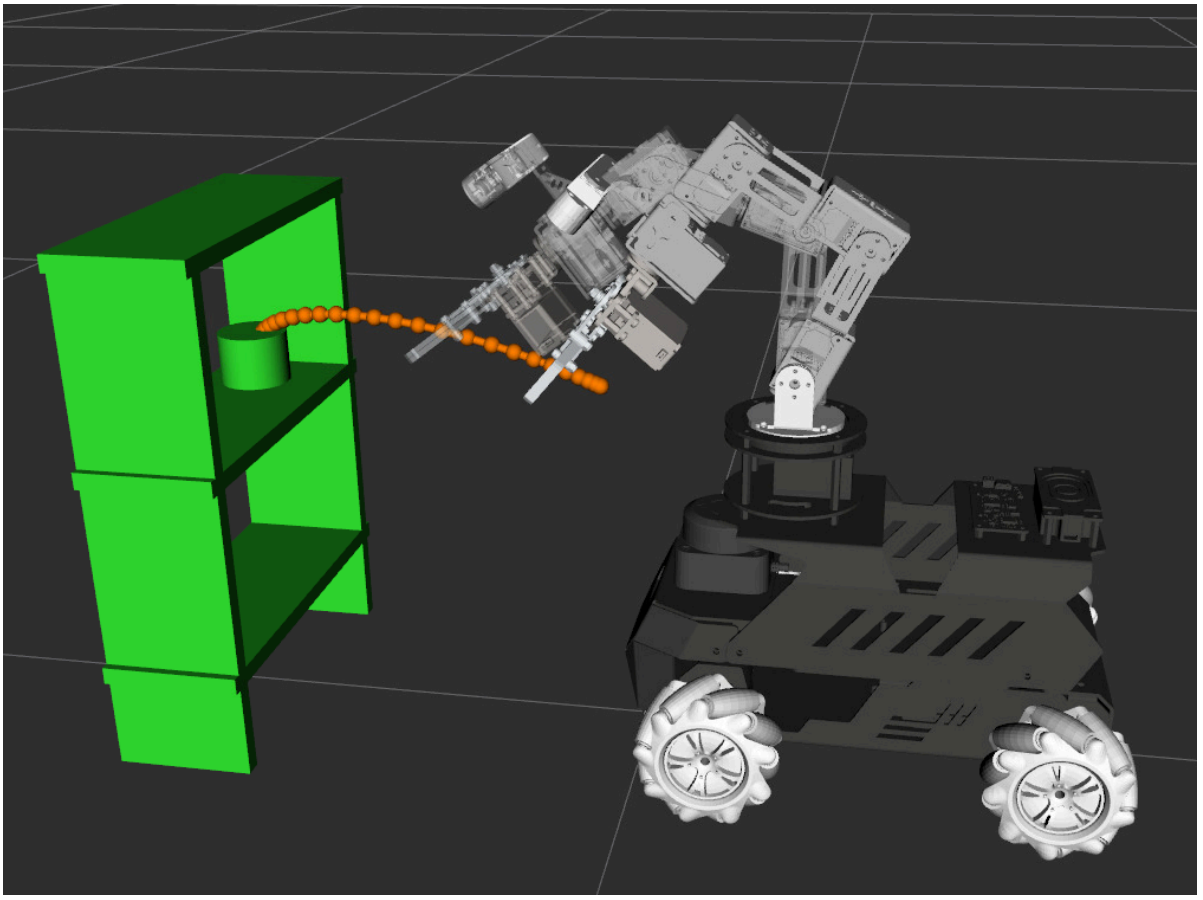


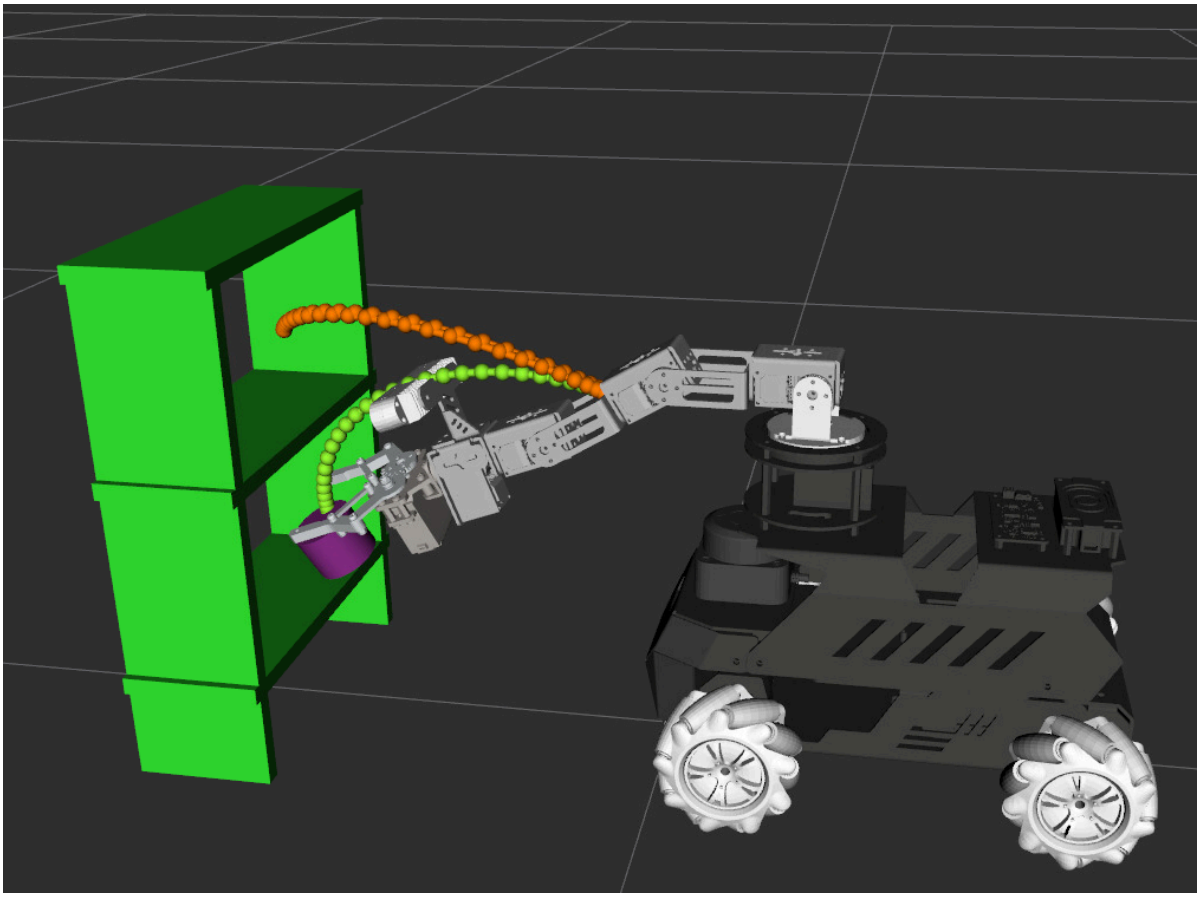
3. Case Demonstration

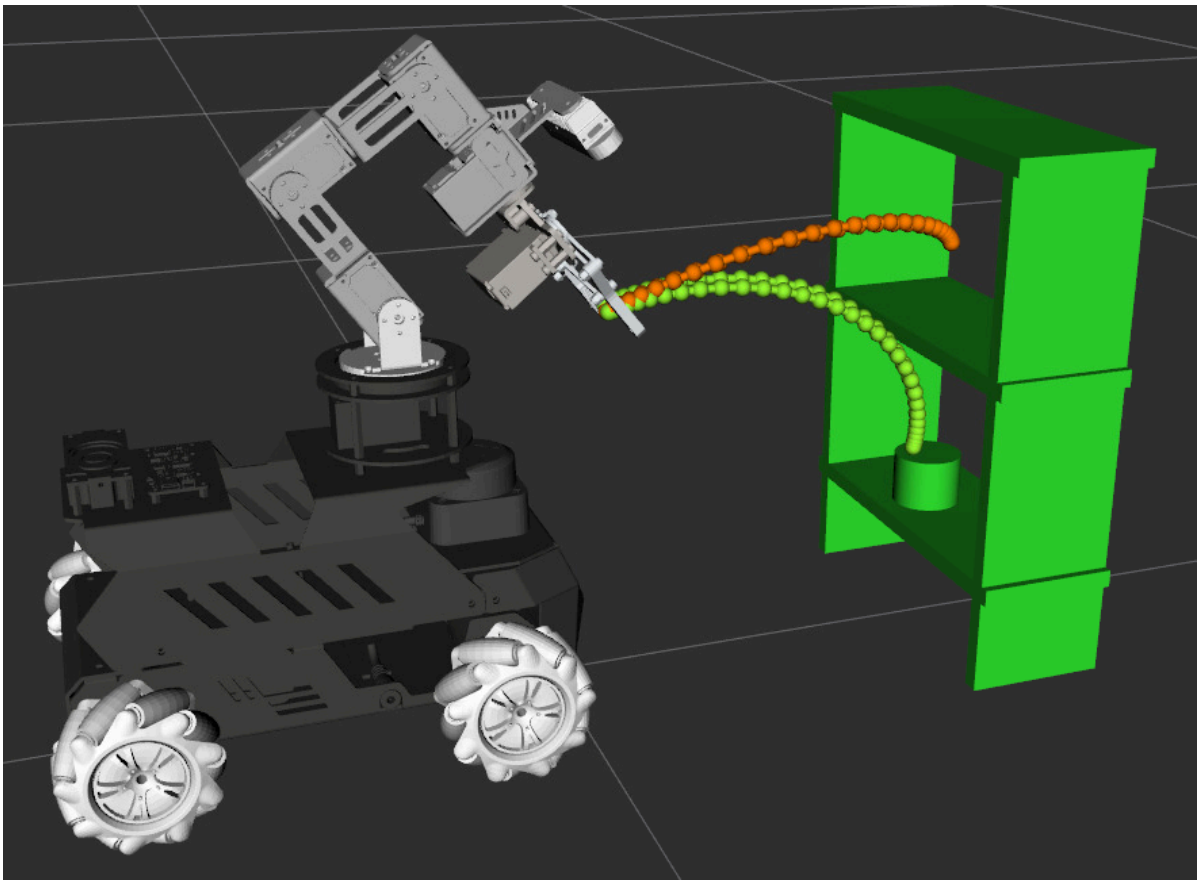
```
ros2 run moveit_demo set_scene
```

After the program runs, the robot arm will plan to the set init pose, then a purple cylindrical object will be added to the gripper. Next, it starts planning and executing the simulated action of placing the object between the middle two layers: first grasp the cylinder from the upper shelf, then the robot arm returns to the initial position -> plans a trajectory to move to the lower shelf to place

the object -> finally the robot arm returns to the initial pose.







4. Core Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/moveit_demo/src/set_scene.cpp
```

Loading the scene file and adding the cylinder:

- `ament_index_cpp::get_package_share_directory("moveit_demo")`: Gets the installation path of the moveit_demo package.
- `loadSceneFile(scene_path)`: Loads the shape.scene scene file and parses out the collision object list.
- Position adaptation: Maps the x coordinate in the scene model to `pose.position.x = 0.26`, `pose.position.y = -old_x`, adjusting the scene model to the front of the robot.
- Orientation adaptation: Rotates the orientation of the scene model by 90° (about the Z axis) through quaternion multiplication, aligning it with the robot coordinate frame.
- `addCollisionObjects(scene_objects)`: Adds all collision objects in the scene to the planning scene.
- Cylinder pre-placement: Adds a cylinder with a height of 0.03 m and a radius of 0.02 m at the middle shelf position (0.24, 0.0, 0.22).
- `applyCollisionObject(cylinder_pre)`: Applies the cylinder collision object to the scene.

Allowed Collision Matrix (ACM) configuration:

- `create_publisher<PlanningScene>("/planning_scene", 10)`: Creates a planning scene publisher, used to send ACM updates.
- `getLinkModelNames()`: Gets the names of all links of the robot.
- `gripper_links = {"llink2", "rlink2", "arm5_Link"}`: The gripper link list, used to re-enable collision detection with base_link.
- `shelf_exempt = {"llink2", "rlink2"}`: The links exempted from collision detection with the shelf, allowing the gripper to touch the shelf edges.
- `shelf_ids = {"Box_0"..."Box_4"}`: The list of shelf collision object IDs.
- `entry_names`: Builds the ACM entry name list, including robot links + cylinder + shelf collision objects.
- `allowed_collision_matrix.entry_values[i].enabled[j] = false`: Sets that collision between the two objects is allowed (collision detection is ignored).
- Between robot links: disables self-collision, but re-enables collision detection between the gripper and base_link.
- Between the shelf and the gripper: collision between llink2/rlink2 and the shelf is allowed (exempted), and collision detection between other links and the shelf is disabled.

Execution flow:

- **Step 1**: Move to the init initial pose.
- **Step 2**: Move to the middle shelf position through a joint-space target, specifying each joint angle (radians).
- **Step 4**: `attachObject("cylinder", "gripper_tcp", touch_links)`: Attaches the cylinder to the gripper; touch_links specifies the links allowed to make contact.
- **Step 4.5**: Retreat to the init pose to prepare for the lower shelf motion.
- **Step 5**: Move to the lower shelf position through a joint-space target.
- **Step 6**: Place the cylinder — first remove it from the scene, then re-add it at the lower shelf position (0.24, 0.0, 0.09).
- `removeCollisionObjects(to_remove)`: Removes the cylinder from the scene.
- `detachObject("cylinder")`: Releases the attachment relationship between the cylinder and the gripper.
- `applyCollisionObject(cyl_placed)`: Re-adds the cylinder at the lower shelf position.
- **Step 7**: Move to the init pose; the task is complete.

```
void initialize()
{
  move_group_interface_ =
    std::make_shared<moveit::planning_interface::MoveGroupInterface>(
      shared_from_this(), "arm_group");
  planning_scene_interface_ =
    std::make_shared<moveit::planning_interface::PlanningSceneInterface>();

  move_group_interface_->setNumPlanningAttempts(30);
  move_group_interface_->setPlanningTime(20.0);
  move_group_interface_->setGoalPositionTolerance(0.02);
```

```
    move_group_interface_->setGoalJointTolerance(0.05);  // wider goal region for
joint-space plans
    move_group_interface_->setWorkspace(-0.1, -0.4, -0.05, 0.5, 0.4, 0.7);
    move_group_interface_->setMaxVelocityScalingFactor(1.0);
    move_group_interface_->setMaxAccelerationScalingFactor(1.0);
    move_group_interface_->setEndEffectorLink("gripper_tcp");
    move_group_interface_->setPoseReferenceFrame("measure");

    const moveit::core::JointModelGroup* joint_model_group =
      move_group_interface_->getRobotModel()->getJointModelGroup("arm_group");
    visual_tools_ = std::make_shared<moveit_visual_tools::MoveItVisualTools>(
      shared_from_this(), "measure", "set_scene",
      move_group_interface_->getRobotModel());
    visual_tools_->deleteAllMarkers();
    visual_tools_->loadRemoteControl();

    // Load shape.scene and cylinder
    try
    {
      std::string scene_path =
        ament_index_cpp::get_package_share_directory("moveit_demo")
        + "/scene/shape.scene";
      RCLCPP_INFO(this->get_logger(),
        "Loading scene file: %s", scene_path.c_str());
      std::vector<moveit_msgs::msg::CollisionObject> scene_objects =
        loadSceneFile(scene_path);
      for (auto& obj : scene_objects)
      {
        for (auto& pose : obj.primitive_poses)
        {
          //Special operations, adapted to scene models
          double old_x = pose.position.x;
          pose.position.x =  0.26;
          pose.position.y = -old_x;

          double qr_x = 0.0, qr_y = 0.0, qr_z = -0.70710678, qr_w = 0.70710678;
          double qo_x = pose.orientation.x;
          double qo_y = pose.orientation.y;
          double qo_z = pose.orientation.z;
          double qo_w = pose.orientation.w;
          pose.orientation.x = qr_w * qo_x + qr_x * qo_w + qr_y * qo_z - qr_z *
qo_y;
          pose.orientation.y = qr_w * qo_y - qr_x * qo_z + qr_y * qo_w + qr_z *
qo_x;
          pose.orientation.z = qr_w * qo_z + qr_x * qo_y - qr_y * qo_x + qr_z *
qo_w;
          pose.orientation.w = qr_w * qo_w - qr_x * qo_x - qr_y * qo_y - qr_z *
qo_z;
        }
      }

      planning_scene_interface_->addCollisionObjects(scene_objects);

      // Pre-place cylinder at middle shelf
      {
        moveit_msgs::msg::CollisionObject cylinder_pre;
```

```
        cylinder_pre.id = "cylinder";
        cylinder_pre.header.frame_id = "measure";
        cylinder_pre.operation = cylinder_pre.ADD;
        shape_msgs::msg::SolidPrimitive cyl_prim;
        cyl_prim.type = cyl_prim.CYLINDER;
        cyl_prim.dimensions.resize(2);
        cyl_prim.dimensions[cyl_prim.CYLINDER_HEIGHT] = 0.03;
        cyl_prim.dimensions[cyl_prim.CYLINDER_RADIUS] = 0.02;

        geometry_msgs::msg::Pose cyl_pose;
        cyl_pose.orientation.w = 1.0;
        cyl_pose.position.x = 0.24;
        cyl_pose.position.y = 0.0;
        cyl_pose.position.z = 0.22;
        cylinder_pre.primitives.push_back(cyl_prim);
        cylinder_pre.primitive_poses.push_back(cyl_pose);
        planning_scene_interface_->applyCollisionObject(cylinder_pre);
      }
    }
    catch (const std::exception& e)
    {
      RCLCPP_ERROR(this->get_logger(),
        "Failed to load scene file: %s", e.what());
    }
    //  Disable robot self-collision.  Only re-enable gripper ↔ base_link
    //  Also exempt llink2/rlink2 ↔ shelf so gripper fingers can touch shelf
edges.
    {
      auto acm_pub = this->create_publisher<moveit_msgs::msg::PlanningScene>(
        "/planning_scene", 10);
      rclcpp::sleep_for(std::chrono::milliseconds(300));

      const auto& links =
        move_group_interface_->getRobotModel()->getLinkModelNames();
      const size_t robot_n = links.size();

      // Gripper links checked against base_link
      const std::vector<std::string> gripper_links = {
        "llink2", "rlink2", "arm5_Link"};

      // Gripper links exempted from shelf collision
      const std::vector<std::string> shelf_exempt = {"llink2", "rlink2"};

      const std::vector<std::string> shelf_ids = {
        "Box_0", "Box_1", "Box_2", "Box_3", "Box_4"};

      // Build entry list: robot links + cylinder + shelf boxes
      std::vector<std::string> entry_names = links;
      entry_names.push_back("cylinder");
      const size_t cyl_idx = entry_names.size() - 1;
      for (const auto& sid : shelf_ids)
        entry_names.push_back(sid);
      const size_t total = entry_names.size();

      // Build name → index map
      std::map<std::string, size_t> name_to_idx;
```

```
      for (size_t i = 0; i < total; i++)
        name_to_idx[entry_names[i]] = i;

      auto it_base = name_to_idx.find("base_link");
      const size_t base_idx = (it_base != name_to_idx.end()) ? it_base->second :
0;

      moveit_msgs::msg::PlanningScene ps_msg;
      ps_msg.is_diff = true;
      ps_msg.allowed_collision_matrix.entry_names = entry_names;
      ps_msg.allowed_collision_matrix.entry_values.resize(total);

      // --- Row 0..robot_n-1 (robot links) ---
      for (size_t i = 0; i < robot_n; i++)
      {
        auto& row = ps_msg.allowed_collision_matrix.entry_values[i];
        row.enabled.resize(total, true);
        // Check robot ↔ shelf (except exempt links)
        bool is_exempt = false;
        for (const auto& ename : shelf_exempt)
          if (entry_names[i] == ename) { is_exempt = true; break; }
        if (!is_exempt)
        {
          for (const auto& sid : shelf_ids)
          {
            auto it = name_to_idx.find(sid);
            if (it != name_to_idx.end())
              row.enabled[it->second] = false;
          }
        }
      }

      // --- Cylinder row ---
      {
        auto& row = ps_msg.allowed_collision_matrix.entry_values[cyl_idx];
        row.enabled.resize(total, true);
      }

      // --- Shelf rows ---
      for (const auto& sid : shelf_ids)
      {
        auto it_s = name_to_idx.find(sid);
        if (it_s == name_to_idx.end()) continue;
        auto& row = ps_msg.allowed_collision_matrix.entry_values[it_s->second];
        row.enabled.resize(total, false);
        for (const auto& ename : shelf_exempt)
        {
          auto it_e = name_to_idx.find(ename);
          if (it_e != name_to_idx.end())
            row.enabled[it_e->second] = true;
        }
      }

      // Re-enable gripper ↔ base_link (both directions)
      for (const auto& gname : gripper_links)
      {
```

```
        auto it = name_to_idx.find(gname);
        if (it == name_to_idx.end()) continue;
        size_t gi = it->second;
        ps_msg.allowed_collision_matrix.entry_values[gi].enabled[base_idx] =
false;
        ps_msg.allowed_collision_matrix.entry_values[base_idx].enabled[gi] =
false;
      }

      acm_pub->publish(ps_msg);
      rclcpp::sleep_for(std::chrono::milliseconds(500));
      RCLCPP_INFO(this->get_logger(),
        "ACM: self-collision OFF, gripper↔base ON, llink2/rlink2↔shelf OFF. "
        "(%zu links + %zu shelf)",
        robot_n, shelf_ids.size());
    }

    // shelf pick-and-place
    const int max_attempts = 5;
    const double SHELF_X = 0.24;
    const double MIDDLE_Z = 0.23;
    const double LOWER_Z  = 0.09;
    auto exec_named = [&](const std::string& desc, const std::string& target,
                          bool show_trajectory = true,
                          rviz_visual_tools::Colors color =
rviz_visual_tools::LIME_GREEN)
    {
      RCLCPP_INFO(this->get_logger(),
        "=== %s: named target '%s' ===", desc.c_str(), target.c_str());
      for (int a = 1; a <= max_attempts; a++)
      {
        move_group_interface_->setNamedTarget(target);
        moveit::planning_interface::MoveGroupInterface::Plan plan;
        if (move_group_interface_->plan(plan) ==
            moveit::core::MoveItErrorCode::SUCCESS)
        {
          if (show_trajectory)
          {
            visual_tools_->publishTrajectoryLine(plan.trajectory_,
              move_group_interface_->getRobotModel()-
>getLinkModel("gripper_tcp"),
              joint_model_group, color);
            visual_tools_->trigger();
          }
          move_group_interface_->execute(plan);
          return true;
        }
        RCLCPP_WARN(this->get_logger(),
          "%s attempt %d/%d failed", desc.c_str(), a, max_attempts);
      }
      RCLCPP_ERROR(this->get_logger(), "%s failed!", desc.c_str());
      return false;
    };

    auto exec_position = [&](const std::string& desc,
                             double x, double y, double z,
```

```
                              rviz_visual_tools::Colors color =
rviz_visual_tools::BLUE)
    {
      RCLCPP_INFO(this->get_logger(),
        "=== %s: position (%.3f, %.3f, %.3f) ===", desc.c_str(), x, y, z);
      for (int a = 1; a <= max_attempts; a++)
      {
        move_group_interface_->setStartStateToCurrentState();
        move_group_interface_->setPositionTarget(x, y, z);
        moveit::planning_interface::MoveGroupInterface::Plan plan;
        if (move_group_interface_->plan(plan) ==
            moveit::core::MoveItErrorCode::SUCCESS)
        {
          visual_tools_->publishTrajectoryLine(plan.trajectory_,
            move_group_interface_->getRobotModel()->getLinkModel("gripper_tcp"),
            joint_model_group, color);
          visual_tools_->trigger();
          move_group_interface_->execute(plan);
          return true;
        }
        RCLCPP_WARN(this->get_logger(),
          "%s attempt %d/%d failed", desc.c_str(), a, max_attempts);
        rclcpp::sleep_for(std::chrono::milliseconds(200));
      }
      RCLCPP_ERROR(this->get_logger(), "%s failed!", desc.c_str());
      return false;
    };

    //  Move to init pose
    if (!exec_named("Step 1: init pose", "init", false))
    {
      RCLCPP_ERROR(this->get_logger(), "Aborting: init pose failed.");
      rclcpp::shutdown();
      return;
    }

    // --- 4b. Move to middle shelf via joint-space target (orange) ---
    {
      RCLCPP_INFO(this->get_logger(),
        "=== Step 2: approach middle shelf via joint target ===");

      std::map<std::string, double> middle_joints;
      middle_joints["arm1"] = 1.5825;
      middle_joints["arm2"] = 0.2489;
      middle_joints["arm3"] = 0.9974;
      middle_joints["arm4"] = 1.9240;
      middle_joints["arm5"] = 1.57;

      bool ok = false;
      for (int a = 1; a <= max_attempts; a++)
      {
        move_group_interface_->setStartStateToCurrentState();
        move_group_interface_->setJointValueTarget(middle_joints);
        moveit::planning_interface::MoveGroupInterface::Plan plan;
        if (move_group_interface_->plan(plan) ==
            moveit::core::MoveItErrorCode::SUCCESS)
```

```
        {
          visual_tools_->publishTrajectoryLine(plan.trajectory_,
            move_group_interface_->getRobotModel()->getLinkModel("gripper_tcp"),
            joint_model_group, rviz_visual_tools::ORANGE);
          visual_tools_->trigger();
          move_group_interface_->execute(plan);
          ok = true;
          break;
        }
        RCLCPP_WARN(this->get_logger(),
          "Step 2 attempt %d/%d failed", a, max_attempts);
        rclcpp::sleep_for(std::chrono::milliseconds(500));
      }

      if (!ok)
      {
        RCLCPP_ERROR(this->get_logger(), "Aborting: middle shelf approach
failed.");
        rclcpp::shutdown();
        return;
      }
    }

    // Attach cylinder at middle shelf
    {
      RCLCPP_INFO(this->get_logger(),
        "=== Step 4: attach cylinder from middle shelf ===");

      std::vector<std::string> touch_links;
      touch_links.push_back("llink2");
      touch_links.push_back("rlink2");
      move_group_interface_->attachObject("cylinder", "gripper_tcp",
touch_links);

    }

    {
      RCLCPP_INFO(this->get_logger(),
        "=== Step 4.5: retreat to init before lower shelf ===");
      bool retreated = false;
      for (int a = 1; a <= max_attempts; a++)
      {
        move_group_interface_->setNamedTarget("init");
        moveit::planning_interface::MoveGroupInterface::Plan plan;
        if (move_group_interface_->plan(plan) ==
            moveit::core::MoveItErrorCode::SUCCESS)
        {
          visual_tools_->publishTrajectoryLine(plan.trajectory_,
            move_group_interface_->getRobotModel()->getLinkModel("gripper_tcp"),
            joint_model_group, rviz_visual_tools::ORANGE);
          visual_tools_->trigger();
          move_group_interface_->execute(plan);
          retreated = true;
          break;
        }
        RCLCPP_WARN(this->get_logger(),
```

```
        "Retreat to init attempt %d/%d failed", a, max_attempts);
      rclcpp::sleep_for(std::chrono::milliseconds(200));
    }
    if (!retreated)
    {
      RCLCPP_ERROR(this->get_logger(),
        "Cannot retreat to init, lower shelf plan will likely fail.");
    }
  }

  // --- 4e. Move to lower shelf via joint-space target ---
  {
    RCLCPP_INFO(this->get_logger(),
      "=== Step 5: approach lower shelf via joint target ===");

    std::map<std::string, double> lower_joints;
    lower_joints["arm1"] = 1.5837;
    lower_joints["arm2"] = -0.1831;
    lower_joints["arm3"] = 1.1294;
    lower_joints["arm4"] = 1.8359;
    lower_joints["arm5"] = 1.3307;

    bool ok = false;
    for (int a = 1; a <= max_attempts; a++)
    {
      move_group_interface_->setStartStateToCurrentState();
      move_group_interface_->setJointValueTarget(lower_joints);
      moveit::planning_interface::MoveGroupInterface::Plan plan;
      if (move_group_interface_->plan(plan) ==
          moveit::core::MoveItErrorCode::SUCCESS)
      {
        visual_tools_->publishTrajectoryLine(plan.trajectory_,
          move_group_interface_->getRobotModel()->getLinkModel("gripper_tcp"),
          joint_model_group, rviz_visual_tools::LIME_GREEN);
        visual_tools_->trigger();
        move_group_interface_->execute(plan);
        ok = true;
        break;
      }
      RCLCPP_WARN(this->get_logger(),
        "Step 5 attempt %d/%d failed", a, max_attempts);
      rclcpp::sleep_for(std::chrono::milliseconds(500));
    }

    if (!ok)
    {
      RCLCPP_WARN(this->get_logger(),
        "Lower shelf joint target failed, will still try to place cylinder.");
    }
  }

  // --- 4f. Place cylinder at lower shelf (detach, reset to upright) ---
  {
    RCLCPP_INFO(this->get_logger(),
      "=== Step 6: place cylinder at lower shelf ===");
```

```
      std::vector<std::string> to_remove = {"cylinder"};
      planning_scene_interface_->removeCollisionObjects(to_remove);
      rclcpp::sleep_for(std::chrono::milliseconds(200));

      move_group_interface_->detachObject("cylinder");

      // Now add a fresh vertical cylinder at lower shelf
      moveit_msgs::msg::CollisionObject cyl_placed;
      cyl_placed.id = "cylinder";
      cyl_placed.header.frame_id = "measure";
      cyl_placed.operation = cyl_placed.ADD;

      shape_msgs::msg::SolidPrimitive cyl_prim;
      cyl_prim.type = cyl_prim.CYLINDER;
      cyl_prim.dimensions.resize(2);
      cyl_prim.dimensions[cyl_prim.CYLINDER_HEIGHT] = 0.03;
      cyl_prim.dimensions[cyl_prim.CYLINDER_RADIUS] = 0.02;

      geometry_msgs::msg::Pose cyl_pose;
      cyl_pose.orientation.w = 1.0;
      cyl_pose.position.x = SHELF_X;
      cyl_pose.position.y = 0.0;
      cyl_pose.position.z = 0.09;  // bottom shelf (z=0.06) + half height

      cyl_placed.primitives.push_back(cyl_prim);
      cyl_placed.primitive_poses.push_back(cyl_pose);
      planning_scene_interface_->applyCollisionObject(cyl_placed);

    }

    // Return to init pose
    exec_named("Step 7: return to init", "init");

    rclcpp::shutdown();
  }
```